

METRO TICKET GENERATING SYSTEM

IN SERVICENOW

Complete Project Documentation

ServiceNow – Service Catalog, Flow Designer, Fare Automation, QR Code Generation and Notifications

Project	Metro Ticket Generating System
Platform	ServiceNow
Primary Interface	Service Catalog / Service Portal
Core Features	Station Selection → Fare Calculation → Validation → Ticket Generation → QR Code
Automation	Catalog Client Scripts, UI Policies, Process Automation Engine, Notifications

Project Overview

The Metro Ticket Generating System in ServiceNow is a digital solution designed to simplify and automate metro ticket booking. Users can select their source and destination stations, enter travel details, and receive a digital ticket with a QR code. The system automatically calculates fares and manages the ticket generation process using ServiceNow features such as Service Catalog, Flow Designer, Business Rules, Notifications, and Access Controls.

The project is carried out in the following five phases:

Phase 1 – Requirement Analysis and Planning: Defines business objectives, functional scope, stakeholders, and the execution roadmap.

Phase 2 – Backend Development and Configurations: Covers data architecture, automation logic, workflows, and business rules.

Phase 3 – UI/UX Development and Customization: Focuses on interface design, navigation flow, and overall usability.

Phase 4 – Data Migration, Testing and Security: Includes data handling, access control, QA testing, and data integrity validation.

Phase 5 – Deployment, Documentation and Final Presentation: Covers deployment, troubleshooting, documentation, demonstration, innovation, scalability, and future enhancements.

The project aims to provide a fast, secure, user-friendly, and automated metro ticketing experience while demonstrating the practical application of ServiceNow in a real-world transportation scenario.

Phase 1 – Requirement Analysis and Planning

1.1 Business Objectives

The main objective of the Metro Ticket Generating System is to digitize and automate metro ticket booking using ServiceNow, providing a faster, secure, and convenient travel experience.

- Enhance Commuter Convenience: Enable quick ticket booking and QR-based digital ticket generation.
- Increase Operational Efficiency: Automate fare calculation, ticket processing, and generation to reduce manual errors.
- Promote Digital Payments: Support cashless payment methods such as UPI, cards, and mobile wallets.
- Enable Data-Driven Decisions: Use digital ticket data to understand passenger trends and peak travel periods.
- Support Sustainability: Reduce paper usage by replacing physical tickets with digital QR tickets.

1.2 Functional Scope

The system includes the following key functional features:

- Service Catalog Ticket Booking: A ServiceNow Catalog Item allows users to book metro tickets by entering Source, Destination, Passenger Type, and Number of Tickets.
- Automated Fare Calculation: The system automatically calculates the fare based on the selected stations, passenger type, and number of tickets.
- Digital QR Ticket Generation: After successful booking, the system generates a digital ticket with a unique QR code for travel verification.

1.3 Stakeholder Mapping

The key stakeholders involved in the Metro Ticket Generating System are:

- Passengers (End Users): Book tickets, provide travel details, and access digital QR tickets.
- Station Managers: Monitor ticket-related activities and assist passengers when required.

- Metro Operations Team: Manage ticketing operations, fares, routes, and overall system processes.
- IT Administrators: Configure, maintain, secure, and troubleshoot the ServiceNow application.

1.4 Execution Roadmap

The project implementation is planned through the following key milestones:

1. Catalog Form Setup: Create the metro ticket booking form with required variables.
2. Fare Logic Script: Implement automated fare calculation based on journey details.
3. QR Code Integration: Generate unique QR codes for digital tickets.
4. Station Details Mapping: Configure station information and source-destination mapping.
5. UI Pages: Develop a simple and user-friendly ticket booking interface.
6. UAT & Deployment: Perform user acceptance testing, resolve issues, and deploy the completed system.



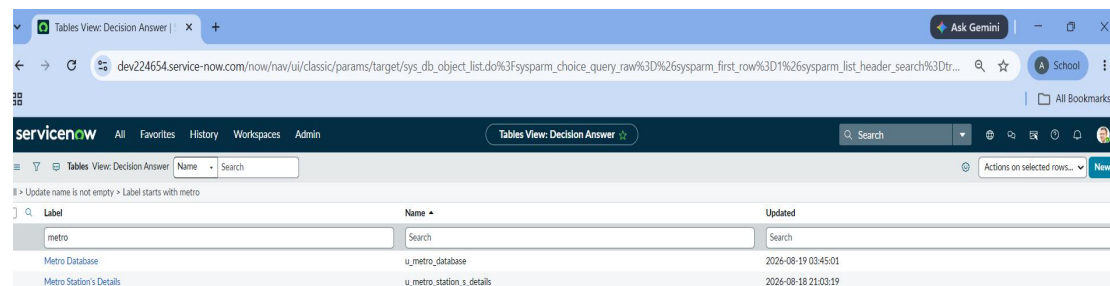
Phase 2-Backend Development and Configurations

2.1 Data Architecture

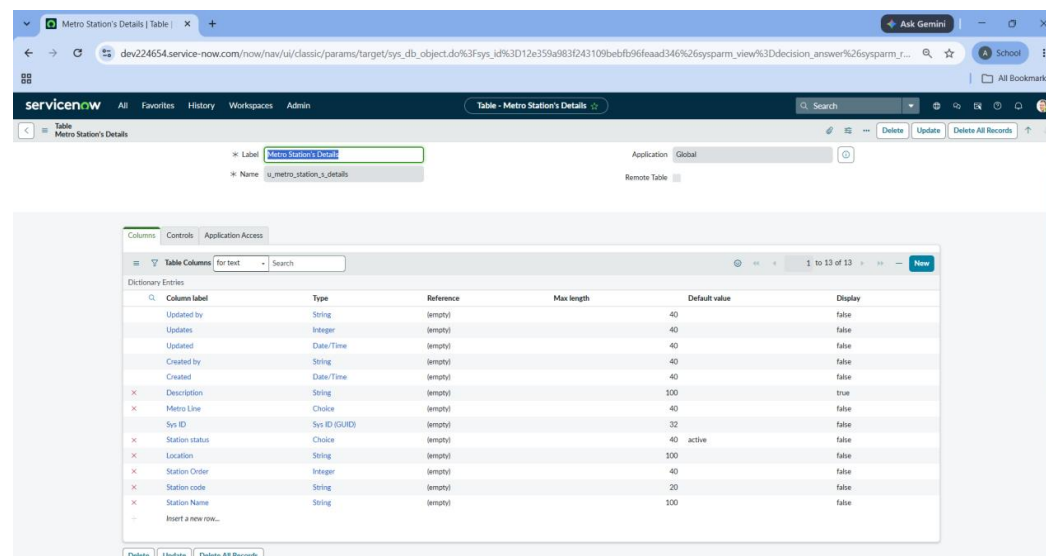
The data architecture defines how data is structured, stored, managed, and used within the Metro Ticket Generating System. In ServiceNow, a dedicated Metro Station Details table is created to maintain important station information required for ticket booking, station mapping, and fare calculation.

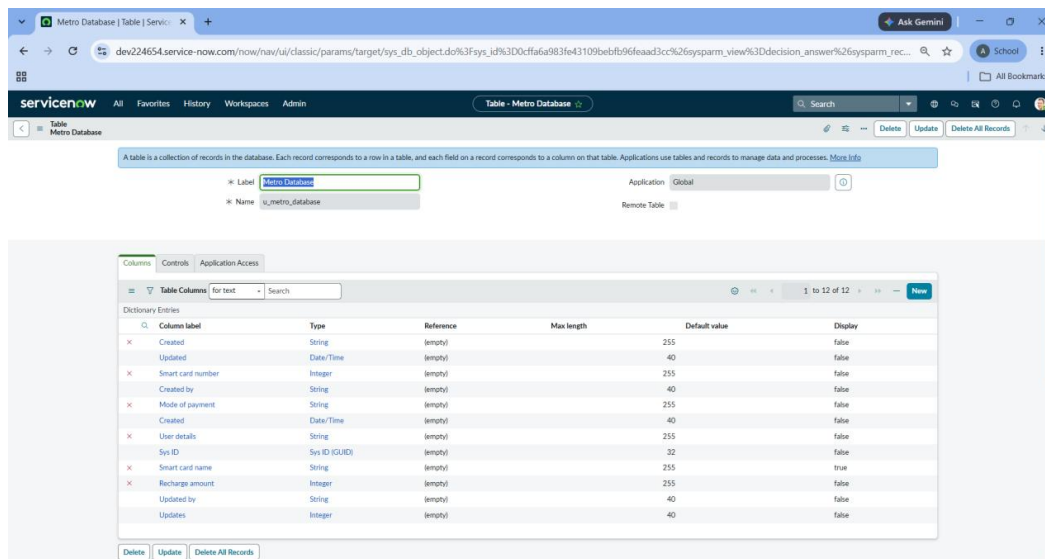
The implementation is carried out through the following activities:

- ✓ Creation of Table: A new Metro Station Details table is created under System Definition → Tables to store station-related information.



- ✓ Creation of Fields: Required fields are added to the table, such as Station Name, Station Code, Location, and other relevant station details.





- ✓ Define Field Properties: Each field is configured with an appropriate data type such as String, Choice, Integer, Reference, or Boolean. Mandatory and read-only properties are applied wherever required.



- ✓ Form Configuration: The created fields are added to the corresponding form using the Form Designer, allowing authorized users to view and manage station information easily.

Description	Location	Metro Line	Station code	Station Name	Station Order	Station status
Rock fort Station	Rock fort	Red line	rf	Rock fort	2	Active
Thanjavur Station	Thanjavur	Blue line	thnj	Thanjavur	5	Active
Thiruvembur Station	Thiruvembur	Blue line	thrv	Thiruvembur	4	Active
Srirangam Station	Srirangam	Red line	Srgm	Srirangam	3	Active
Trichy Metro Station	Trichy	Blue line	try	Trichy	1	Active

- ✓ **Data Management:** The structured station data acts as a reliable source for source-destination selection, fare calculation, ticket processing, and future system expansion.

This structured approach ensures that metro station information is maintained consistently and can be efficiently accessed by different components of the ticketing system.

2.2 Automation Logic

The Automation Logic is implemented using Catalog Client Scripts and Flow Designer to automate the metro ticket booking and fare calculation process. This reduces manual processing and ensures that the ticket information is generated accurately based on the user's selections.

1. Catalog Client Script – Automated Fare Calculation

A Catalog Client Script is configured on the Book A Metro Ticket catalog item. The script runs when the user selects the Type of Journey and automatically calculates the fare based on the selected source, destination, journey type, and number of passengers.

The screenshot shows the configuration for a Catalog Client Script named 'FareCalculation'. The interface includes a header bar with navigation icons, a title 'Catalog Client Scripts FareCalculation', and action buttons 'Update' and 'Delete'. A blue informational banner at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.'

The configuration fields are as follows:

- Name:** FareCalculation
- Applies to:** A Catalog Item
- Active:** ☒
- UI Type:** All
- Application:** Global
- Type:** onChange
- Catalog item:** Book A Metro Ticket
- Variable name:** type_of_journey
- Applies on a Catalog Item view:** ☒
- Applies on Requested Items:** ☐
- Applies on Catalog Tasks:** ☐

The script supports:

- 👉 Automatic fare calculation for different source and destination combinations.
- 👉 Separate calculation for single and return journeys.
- 👉 Automatic multiplication of fare based on the number of passengers.

- 👉 Clearing of irrelevant fare fields when the journey type changes.
- 👉 This provides dynamic fare calculation without requiring manual input from the user.

```

servicenow All Favorites : Catalog Client Scripts - FareCalcula... Search
Script
1 function onChange(control, oldValue, newValue, isLoading) {
2     if (isLoading || newValue == '') {
3         return;
4     }
5     // Get values from the catalog form
6     var source = g_form.getValue('starting_from');
7     var destination = g_form.getValue('going_to');
8     var numPassengers = parseInt(
9         g_form.getValue('no_of_passengers'),
10        10
11    );
12    // Check passenger count
13    if (isNaN(numPassengers) || numPassengers <= 0) {
14
15        g_form.clearValue('amount_for_single_journey');
16        g_form.clearValue('amount_including_return');
17
18        return;
19    }
20    // Variable to store fare per passenger
21    var fare = 0;
22    // TRICHY -> THANJAVUR = ₹20
23    if (
24        source == 'edc50d4a83b247109bebf96fead3c0' &&
25        destination == '72c64dca83b247109bebf96fead380'
26    ) {
27
28        fare = 20;
29    }
30    // TRICHY -> THIRUVERUMBUR = ₹15
31    else if (
32        source == 'edc50d4a83b247109bebf96fead3c0' &&
33        destination == '8f8645ca83b247109bebf96fead381'
34    ) {
35
36        fare = 15;
37    }
38    // TRICHY -> SRIRANGAM = ₹10
39    else if (
40        source == 'edc50d4a83b247109bebf96fead3c0' &&
41        destination == 'a1568d8a83b247109bebf96fead380'
42    ) {
43
44        fare = 10;
45    }
46    // TRICHY -> ROCK FORT = ₹10
47    else if (
48        source == 'edc50d4a83b247109bebf96fead3c0' &&
49        destination == '6216c58a83b247109bebf96fead300'
50    ) {

```

```

servicenow All Favorites : Catalog Client Scripts - FareCalcula... Search
Script
1 function onChange(control, oldValue, newValue, isLoading) {
51
52        fare = 10;
53    }
54    // NO VALID ROUTE
55    if (fare == 0) {
56        g_form.clearValue('amount_for_single_journey');
57        g_form.clearValue('amount_including_return');
58        return;
59    }
60    // SINGLE JOURNEY
61    if (newValue == '1') {
62        var singleAmount = fare * numPassengers;
63        // Display single journey amount
64        g_form.setValue(
65            'amount_for_single_journey',
66            singleAmount.toString()
67        );
68        // Clear return amount
69        g_form.clearValue(
70            'amount_including_return'
71        );
72    }
73    // RETURN JOURNEY
74    else if (newValue == '2') {
75        var singleAmount = fare * numPassengers;
76        var returnAmount = singleAmount * 2;
77        // Display single journey amount
78        g_form.setValue(
79            'amount_for_single_journey',
80            singleAmount.toString()
81        );
82        // Display total return amount
83        g_form.setValue(
84            'amount_including_return',
85            returnAmount.toString()
86        );
87    }
88 }
89

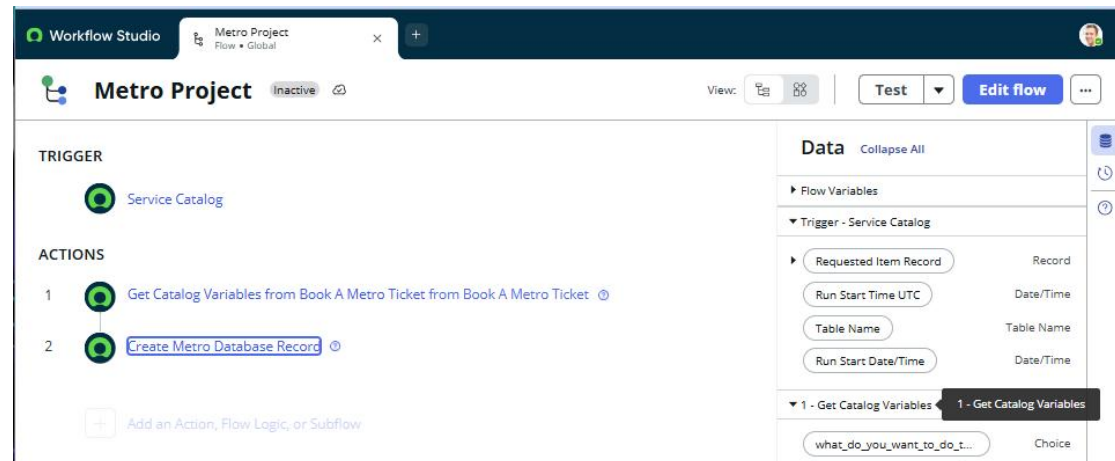
```

Flow Designer – Automated Ticket Processing

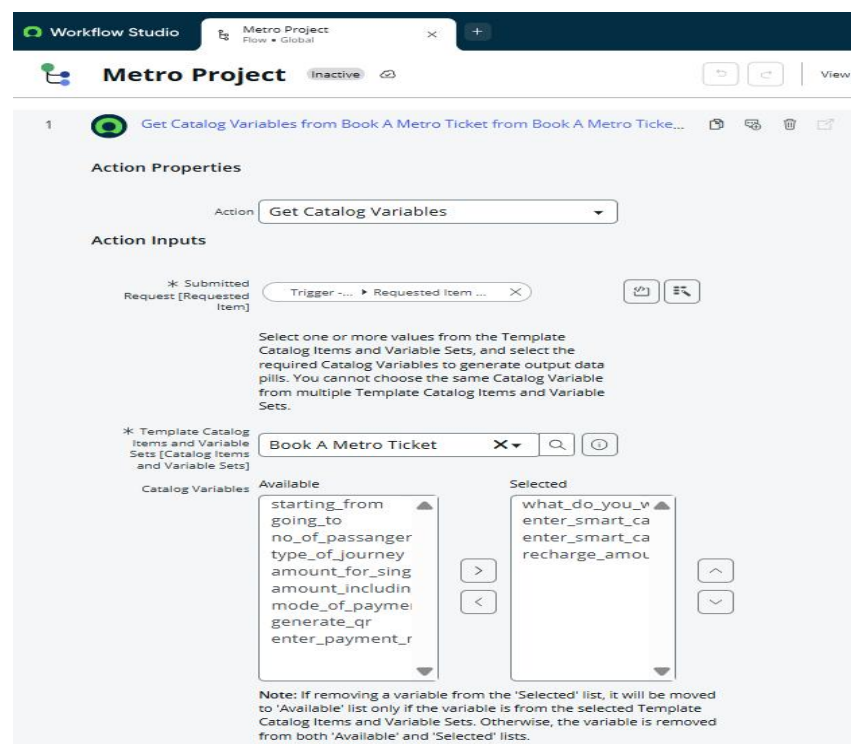
A Flow Designer flow named Metro Project is created to automate the backend ticket processing.

The flow performs the following steps:

- Service Catalog Trigger: The flow starts when a metro ticket request is submitted.



- Get Catalog Variables: Retrieves the required variables from the Book A Metro Ticket catalog item.



- **Create Record:** Creates a new record in the Metro Database table using the submitted ticket information.

The screenshot displays the Workflow Studio interface for the Metro Project. The flow is titled 'Metro Project' and is currently 'Inactive'. The flow consists of two actions:

1. Get Catalog Variables from Book A Metro Ticket from Book A Metro Ticket
2. Create Metro Database Record

The 'Create Metro Database Record' action is selected, and its properties are shown in the 'Action Properties' section. The 'Action' is set to 'Create Record'. The 'Action Inputs' section shows a table with fields:

* Table	* Fields
Metro Database [u_metro_datab...	Mode of payment: BHIM UPI
	Recharge amount: 1 - Get Catal... recharge_am...
	Smart card name: 1 - Get... enter_smart_card...
	Smart card number: 1 - Get... enter_smart_card...
	User details: Trigger - Servi... Create...
	Created (u_created): Trigger - Service... Create...

Buttons for 'Delete', 'Cancel', and 'Done' are located at the bottom right of the interface.

- **Automated Processing:** The flow ensures that the booking information is transferred and stored systematically for further ticket generation and processing.

This automation provides a smooth connection between the Service Catalog, fare calculation, and backend database, improving efficiency and reducing manual intervention.

2.3 Business Rules and Validation Logic

Business rules and client-side configurations are used to control the behavior of the Metro Ticket Booking application. They ensure that fields are displayed correctly, user inputs are validated, and QR codes are generated during the ticket booking process.

Catalog UI Policy – Conditional Fields

A Catalog UI Policy is configured to dynamically display the Enter Payment Mode field when the user selects Others as the Mode of Payment. This improves usability by showing only the fields relevant to the user's selection.

The screenshot shows the 'Catalog UI Policy' configuration page. The title is 'Show Enter Payment Mode when Others is selected'. The 'When to Apply' tab is active. A blue box contains the following conditions:

- 1. The catalog UI policy is Active
- 2. The items in the Conditions field evaluate to true
- 3. The field specified in the catalog UI policy is present on the specified catalog item

Below the conditions, there are buttons for 'Add Filter Condition' and 'Add OR Clause'. The 'Catalog Conditions' section shows a dropdown menu with 'mode_of_payment' selected, followed by 'is' and 'Others'. There are also 'AND', 'OR', and a red 'X' button. Below this, there are checkboxes for 'Applies on a Catalog Item view' (checked), 'Applies on Catalog Tasks' (unchecked), and 'Applies on Requested Items' (unchecked). To the right, there are two blue boxes: the first says 'Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form' with an 'On load' checkbox checked; the second says 'Reverse the effects of the catalog UI policy actions when the Conditions evaluate to false' with a 'Reverse if false' checkbox checked.

Catalog Client Script – QR Code Generation

An OnSubmit Catalog Client Script is implemented to generate a QR code when the ticket request is submitted. The script creates a unique ticket URL using the request identifier and displays the QR code through a modal window. This enables the passenger to access their digital ticket conveniently.

The screenshot shows the 'Catalog Client Scripts - Generate QR' configuration page. The 'Name' field is 'Generate QR'. The 'Applies to' dropdown is 'A Catalog Item'. The 'Active' checkbox is checked. The 'UI Type' dropdown is 'Mobile / Service Portal'. The 'Application' dropdown is 'Global'. The 'Type' dropdown is 'onChange'. The 'Catalog item' dropdown is 'Book A Metro Ticket'. The 'Variable name' field is 'generate_qr'. The 'Applies on a Catalog Item view' checkbox is checked. The 'Applies on Requested Items' checkbox is checked. The 'Applies on Catalog Tasks' checkbox is checked. The 'Script' section contains the following code:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading) {
3     return;
4   }
5   if (newValue != 'true' && newValue != '1') {
6     return;
7   }
8   var from = g_form.getDisplayValue('starting_from');
9   var to = g_form.getDisplayValue('going_to');
10  var passengers = g_form.getDisplayValue('no_of_passengers');
11  var journeyType = g_form.getDisplayValue('type_of_journey');
12  var singleAmount =
13    g_form.getValue('amount_for_single_journey');
14  var returnAmount =
15    g_form.getValue('amount_including_return');
16  var payment =
17    g_form.getDisplayValue('mode_of_payment');
18  var amount = singleAmount;
19  if (JourneyType == 'Return' && returnAmount) {
20    amount = returnAmount;
21  }
22  var qrText =
23    'METRO_TICKET' +
```

Catalog Client Script – Passenger Validation

An OnChange Catalog Client Script is used to validate the Number of Passengers field. If the user enters non-numeric characters, the system displays a validation message and prevents incorrect data from being processed.

The screenshot shows the ServiceNow interface for configuring a Catalog Client Script. The header includes the ServiceNow logo, navigation links (All, Favorites), and a search bar. The main title is 'Catalog Client Scripts - fieldValidation'. Below the title is a blue informational banner about client-script execution in strict mode. The configuration form is divided into two columns. The left column contains fields for 'Name' (fieldValidation), 'Applies to' (A Catalog Item), 'Active' (checked), and 'UI Type' (Desktop). The right column contains fields for 'Application' (Global), 'Type' (onChange), 'Catalog item' (Book A Metro Ticket), and 'Variable name' (no_of_passangers). Below these fields are three checkboxes: 'Applies on a Catalog Item view' (checked), 'Applies on Requested Items' (unchecked), and 'Applies on Catalog Tasks' (unchecked). At the bottom, there is a 'Script' editor with a toolbar and a code editor showing the following JavaScript code:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2
3     if (isLoading || newValue == '') {
4         return;
5     }
6
7
8     if (isNaN(newValue)) {
9         alert('please enter the values in number format only');
10    }
11
12
13
14 }
15
16
17
```

These configurations help maintain data accuracy, dynamic field behavior, and a smooth ticket booking experience.

Phase 3 – UI/UX Development and Customization

3.1 Interface Design

The Interface Design focuses on creating a simple, clear, and user-friendly interface for the Metro Ticket Generating System. A Service Catalog Item is created in ServiceNow to provide passengers with an easy way to enter their journey and ticket details.

➤ Service Catalog Creation

A catalog item named Book A Metro Ticket is created under the Service Catalog. This acts as the main interface through which passengers submit their ticket booking requests.

The screenshot shows the ServiceNow interface for creating a Catalog Item. At the top, there's a header bar with the title 'Catalog Item Book A Metro Ticket' and buttons for 'Copy', 'Try It', 'Update', 'Edit in Catalog Builder', and 'Delete'. Below this is a blue banner with a message: 'Build and modify items faster with the improved Catalog Builder.' followed by a close button. A light blue box contains instructions: 'Catalog items are goods or services available to order from the service catalog. Items can be anything from hardware, like tablets and phones, to software applications, to furniture and office supplies.' followed by two bullet points: 'Enter a Name and Short description to display for the item.' and 'Enter a Price, approvals, variables, and other information as needed.' The form fields are arranged in two columns. The left column includes: 'Name' (Book A Metro Ticket), 'Catalogs' (Service Catalog), 'Category' (Services), 'State' (-- None --), 'Checked out' (-- None --), and 'Owner' (System Administrator). The right column includes: 'Application' (Global), 'Active' (checked), and 'Fulfillment automation level' (Unspecified).

➤ Variables Configuration

Different variables are configured based on the information required for ticket booking. These variables allow users to enter or select details such as:

1. Source Station
2. Destination Station
3. Passenger Type
4. Number of Tickets
5. Type of Journey
6. Mode of Payment

Variables (13)

Variable Sets

Catalog UI Policies (6)

Catalog Client Scripts (5)

Available For

Not Available For

Categories (1)

Catalogs (1)

Catalog Data Lookup Definitions

Related Articles

Related Catalog Items

Assigned Topics

≡

🔍

Order

Search

⌂

—

Actions on selected rows...

New

Catalog item = Book A Metro Ticket

☐	Type	Question	Order
☐	Multiple Choice	What do you want to do today ?	100
☐	Single Line Text	Enter smart card number	200
☐	Single Line Text	Enter smart card name	300
☐	Single Line Text	Recharge amount	350
☐	Reference	Starting from ?	500
☐	Reference	Going to ?	600
☐	Single Line Text	No of passangers	700
☐	Multiple Choice	Type of Journey	800
☐	Single Line Text	Amount for single journey	820
☐	Single Line Text	Amount including return	840
☐	Select Box	Mode of payment	900
☐	CheckBox	Generate QR	1,000
☐	Single Line Text	Enter payment mode	1,000

<<

<

1

>

>>

to 13 of 13

Fields can be configured as Choice, String, Reference, or Multi-line Text and can be made mandatory or read-only based on the requirement.

➤ User-Friendly Design

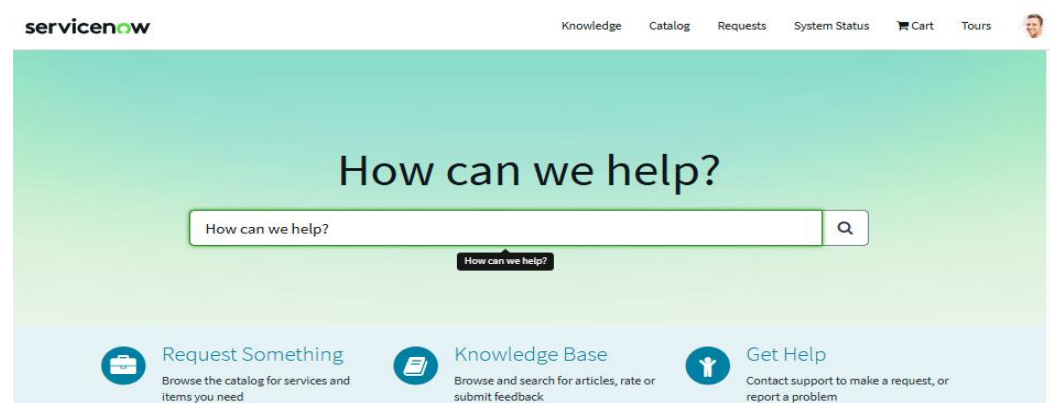
Tooltips, example text, mandatory fields, and dependent variables are configured to guide users while completing the form. This ensures that the booking interface is simple, organized, and easy to navigate, reducing incorrect data entry.

3.2 Navigation Flow

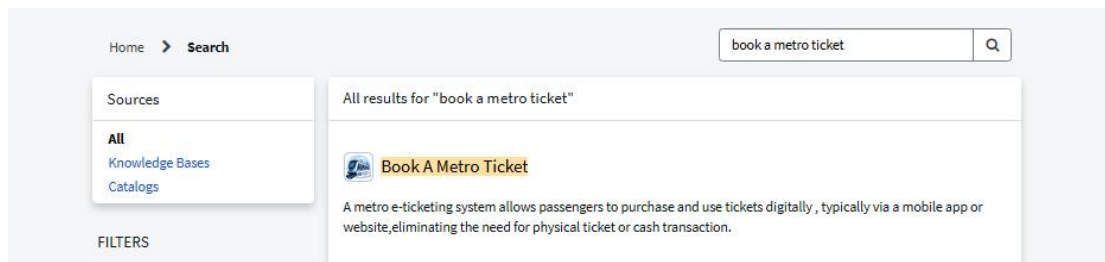
The navigation flow defines how users access and complete the metro ticket booking process in ServiceNow. The process is designed to provide a simple flow from opening the service portal to submitting a ticket request.

➤ Navigation Process

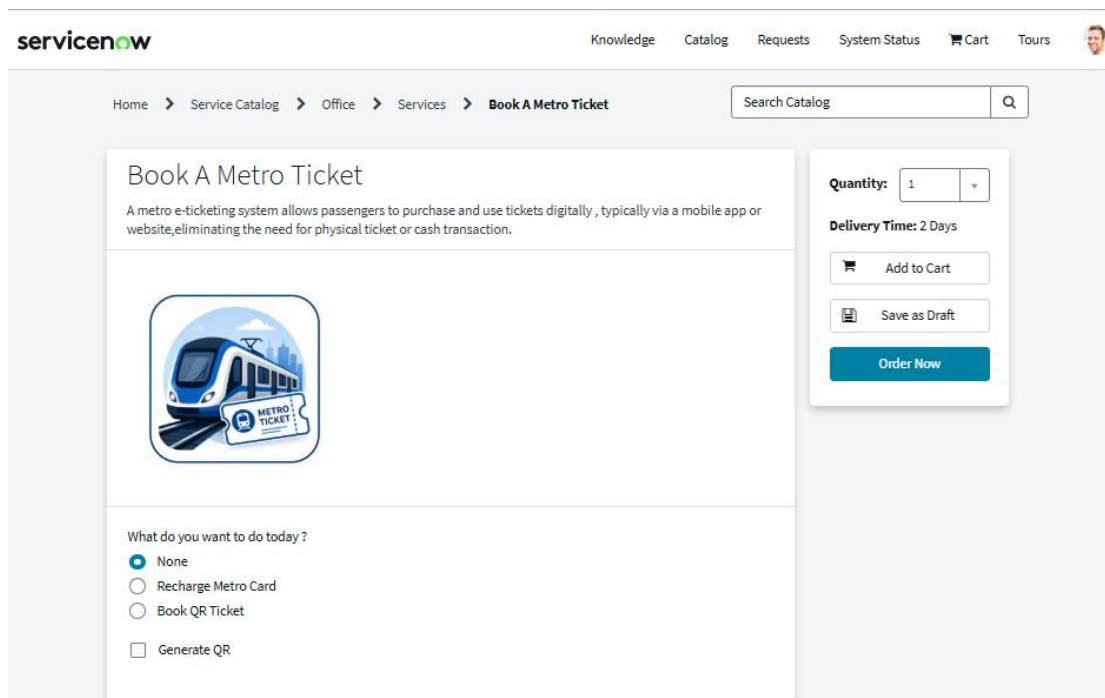
Access Service Portal: Log in to the ServiceNow instance and open the Service Portal.



Search Catalog Item: Search for Book A Metro Ticket in the Service Catalog.



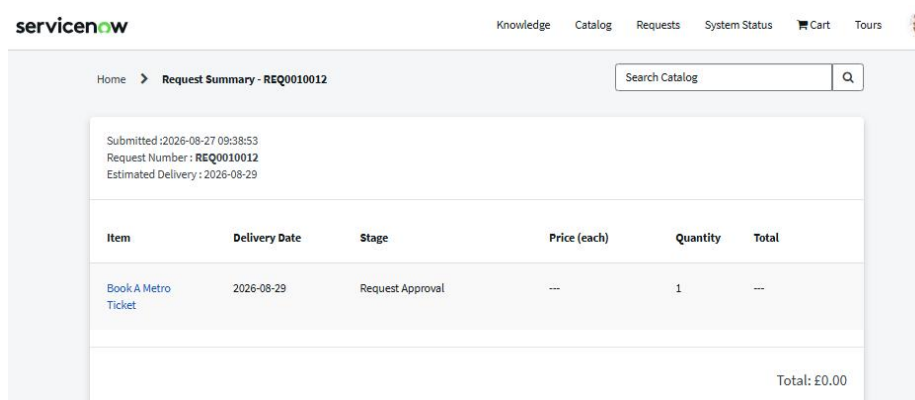
Enter Details: Fill in the required journey, passenger, and payment details.



Submit Request: Click Submit to create the metro ticket request.

Request Generation: ServiceNow generates a unique Request Number for the submitted booking.

Requested Item: The submitted ticket details can be viewed under the corresponding Requested Item (RITM).



This navigation flow provides a clear and straightforward booking experience, allowing users to complete their ticket request with minimal steps.

3.3 Usability

The Metro Ticket Generating System is designed with a strong focus on usability, ensuring that users can book tickets quickly and efficiently through a simple and intuitive interface. The catalog form provides clear labels, dropdown selections, and automated field population to minimize user effort and reduce input errors.

A key usability feature is the real-time fare preview, where the ticket amount is automatically calculated and displayed based on the selected source station, destination station, journey type, and number of passengers. This allows users to review the fare before submitting the request, improving transparency and user confidence.

Additional usability enhancements include:

- a) Simple and user-friendly Service Portal interface.
- b) Dropdown-based station selection to reduce manual entry.
- c) Automatic fare calculation without user intervention.
- d) Dynamic field visibility based on user selections.
- e) QR-based digital ticket generation for convenient access and travel.

These features provide a smooth and efficient booking experience while reducing the possibility of incorrect submissions.

The screenshot displays a web form for booking a Metro QR ticket. At the top, it asks 'What do you want to do today?' with three radio button options: 'None', 'Recharge Metro Card', and 'Book QR Ticket' (which is selected). Below this, the form is divided into several sections. The first section, '* Starting from ?', contains a dropdown menu with 'Trichy Metro Station' selected. The second section, '* Going to ?', has a dropdown menu with 'Thanjavur Station' selected. The third section, '* No of passangers', features a text input field with the value '3'. The fourth section, '* Type of Journey', includes three radio button options: 'None', 'Single', and 'Return' (which is selected). Below these are two read-only text boxes: 'Amount for single journey' showing '60' and 'Amount including return' showing '120'. The final section, '* Mode of payment', has a dropdown menu with 'Card' selected. The form is styled with a light blue header and footer, and a white main content area with a light blue border.

Phase 4 – Data Migration, Testing and Security

4.1 Data Handling

The Data Handling process ensures that ticket booking information submitted through the Service Catalog is securely captured and stored in the custom Metro Database table. The system uses Process Automation through Flow Designer to transfer and map catalog variables into structured database records.

The process includes:

1. **Capture Catalog Data:** User-entered ticket details are captured from the Book A Metro Ticket catalog item.
2. **Get Catalog Variables:** The Flow Designer Get Catalog Variables action retrieves the submitted variables.
3. **Create/Update Record:** A record is created or updated in the Metro Database table.
4. **Variable Mapping:** The Add Fields (+) option is used to map each catalog variable to its corresponding field in the Metro Database.
5. **Structured Storage:** The mapped information is stored systematically for ticket tracking, processing, and future reference.

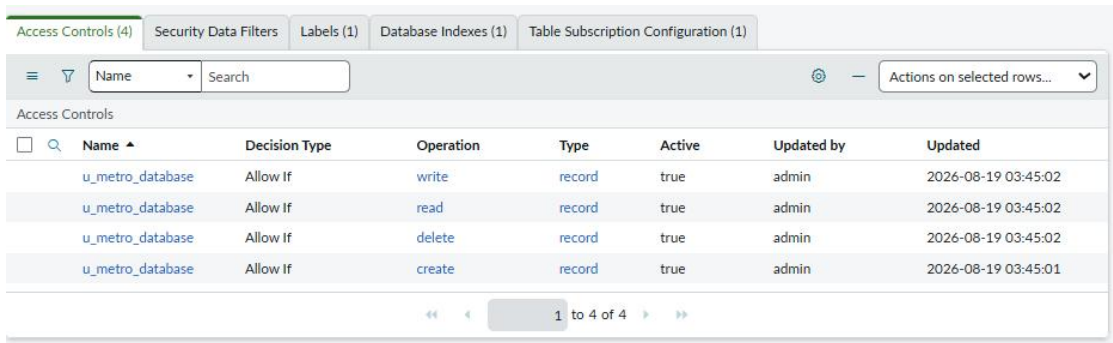
This automated approach ensures accurate, consistent, and efficient handling of ticket data while reducing manual data entry.

4.2 Access Control

Role-Based Access Control (RBAC) is implemented using Access Control Lists (ACLs) to protect the Metro Database and restrict unauthorized access to stored ticket information.

- **Passenger/User Access:** Users can directly book tickets and recharge their metro cards through the Service Catalog but cannot directly access the Metro Database table.
- **Database Protection:** The Metro Database is protected using ServiceNow ACLs to prevent unauthorized viewing or modification of records.
- **Default ACLs:** When the custom table is created, ServiceNow automatically generates the default table-level ACLs. The project uses these default ACL configurations.

- **Role-Based Permissions:** Access to sensitive data and database operations is controlled based on the user's assigned role.



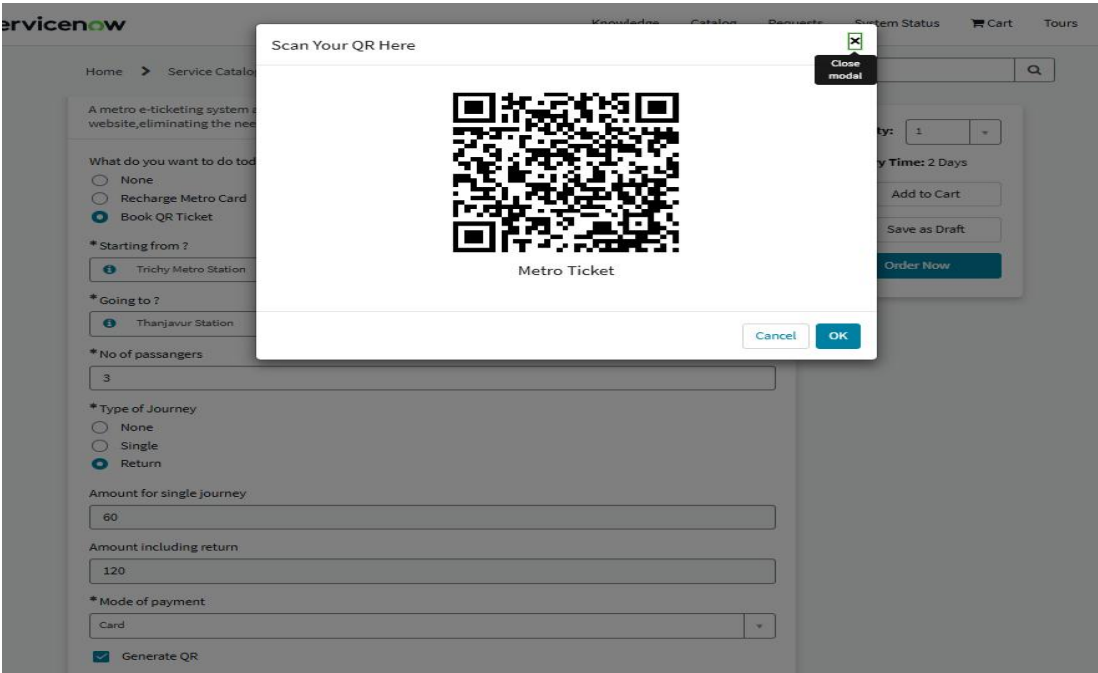
The screenshot shows a table titled 'Access Controls' with 8 columns: Name, Decision Type, Operation, Type, Active, Updated by, and Updated. There are 4 rows of data, all with 'u_metro_database' as the Name and 'Allow If' as the Decision Type. The Operations are 'write', 'read', 'delete', and 'create' respectively. All are active and updated by 'admin' on 2026-08-19. The table is part of a larger interface with tabs for 'Access Controls (4)', 'Security Data Filters', 'Labels (1)', 'Database Indexes (1)', and 'Table Subscription Configuration (1)'.

Name	Decision Type	Operation	Type	Active	Updated by	Updated
u_metro_database	Allow If	write	record	true	admin	2026-08-19 03:45:02
u_metro_database	Allow If	read	record	true	admin	2026-08-19 03:45:02
u_metro_database	Allow If	delete	record	true	admin	2026-08-19 03:45:02
u_metro_database	Allow If	create	record	true	admin	2026-08-19 03:45:01

This approach ensures that passengers can use the required services without having direct access to backend database records, improving data security and integrity.

4.3 QA Testing – Catalog Form, QR Generation and Visibility

QA testing was conducted to verify the correct functioning of the Metro Ticket Generating System. The Service Catalog form was tested for proper field loading, mandatory field validation, station selection, passenger details, and fare calculation. The conditional field visibility was tested to ensure that relevant fields appear or hide based on the user's selection. The QR generation was also tested after submitting the ticket to confirm that a valid QR code is generated with the required booking details. These tests helped ensure that the ticket booking process works correctly, provides proper field visibility, and generates the digital QR ticket successfully.



Metro Databases

User details

Search

Actions on selected rows...

New

All

User details

Smart card name

Smart card number

Recharge amount

Mode of payment

Created

Search

Search

Search

Search

Search

Search

Admin

Rupay

90,109

1,000

card

20/08/26

4.4 Field Validation and Automation

Field validation was implemented to ensure that users provide complete and valid booking details. The source station and destination station fields were validated to prevent empty submissions, while the fare amount was checked against the selected travel details. The travel date was also validated to ensure that only approved dates can be selected.

- **Mandatory Fields:** Required fields were enforced using client-side and server-side validation to prevent incomplete ticket submissions.
- **Auto-Populate Logic:** Default values such as Single Journey or Return Journey were automatically set using Catalog Client Scripts and UI Policies.
- **Field Mapping:** The Process Automation Engine was used to map user-provided values between the relevant fields and automatically populate the required information, maintaining data integrity.

Phase 5 – Deployment, Documentation and Final Presentation

5.1 Troubleshooting

During the deployment and testing stage, identified issues were analyzed and resolved to ensure smooth functioning of the Metro Ticket Generating System. Issues related to form visibility, fare calculation, field validation, QR code generation, and automation were tested and corrected. Troubleshooting helped improve the reliability and consistency of the application.

5.2 Adherence to Timelines

The project was developed according to a planned execution roadmap covering requirement analysis, backend configuration, UI development, testing, and final deployment. Each phase was completed systematically to ensure that the major functionalities were implemented within the planned schedule.

5.3 Innovation

The project introduces a digital approach to metro ticket booking using ServiceNow Service Catalog, automated fare calculation, and QR-based digital ticket generation. The combination of automated processing and QR tickets reduces manual effort and provides users with a faster and more convenient ticketing experience.

5.4 Functional Overview

The system allows users to select source and destination stations, enter passenger and journey details, calculate the applicable fare, submit the booking, and receive a digital QR-based ticket. ServiceNow components such as Service Catalog, Client Scripts, UI Policies, Process Automation, and Notifications work together to automate the complete ticket booking process.

5.5 Technical Blueprint

The technical design consists of the ServiceNow Service Catalog as the user interface, catalog variables for collecting travel details, scripts and UI policies for form behavior, backend data structures for storing information, automation for processing requests, and QR generation for digital ticket delivery.

5.6 Setup Manual

A setup manual was prepared to describe the steps required to configure and run the application. It includes ServiceNow instance setup, catalog item configuration, variable creation, client scripts, UI policies, data configuration, automation, QR generation, and testing procedures.

5.7 Project Demo Video Planning

The project demonstration video is planned to present the complete user journey. The demonstration begins with opening the metro ticket catalog item, entering travel details, viewing the calculated fare, submitting the request, and displaying the generated QR ticket.

5.8 Visual Demonstration

The visual demonstration showcases the actual working system through screenshots of the catalog form, station selection, fare calculation, conditional fields, ticket submission, and QR code generation. This provides clear evidence of the implemented functionalities.

5.9 Explanation Clarity

The project functionalities were presented in a clear and systematic manner. The demonstration explains the purpose of the system, booking workflow, automated fare calculation, field validation, QR generation, and the role of ServiceNow components.

5.10 Realism and Quality

The system was designed to represent a realistic metro ticket booking environment. Realistic station selection, passenger information, journey types, fare calculation, digital ticket generation, and user interactions were incorporated to make the application practical and easy to understand.

Screenshot: Final polished catalog form and QR ticket.

5.11 Scalability and Future Plan

The system can be further enhanced to support multiple metro lines, larger station databases, real-time train information, online payment integration, ticket cancellation,

ticket history, mobile access, and advanced passenger notifications. The architecture can also be extended to support larger numbers of users and automated metro service operations.

Project Conclusion

The Metro Ticket Generating System on ServiceNow provides a centralized and automated solution for metro ticket booking and digital ticket generation. The system simplifies the booking process through station selection, automated fare calculation, field validation, and QR-based ticket generation.

By using Service Catalog, automation, business rules, and notifications, the system reduces manual effort and improves accuracy, efficiency, and user experience. Its scalable design also provides a strong foundation for future enhancements such as online payments, ticket cancellation, real-time metro information, and mobile integration.

Overall, the project demonstrates how ServiceNow can be effectively used to build a secure, efficient, user-friendly, and scalable digital metro ticketing solution.